LAPORAN PRAKTIKUM
APLIKASI KOMPUTER DAN BASIS DATA

CONSTRUCTOR DAN OVERLOADING



Praktikan
Elsadanika Maura Sitorus
211344013 – 2NK

PROGRAM STUDI TEKNIK TELEKOMUNIKASI
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI BANDUNG
2023

Praktikum

Constructor dan Overloading

TUJUAN PRAKTIKUM

- Mahasiswa mampu memahami penggunaan constructor dalam bahasa java.
- Mahasiswa dapat memahami penggunaan method dalam bahasa java.
- Mahasiswa dapat memahami penggunaan overloading dalam bahasa java.

TEORI DASAR

A. Constructor

Constructor adalah sebuah metode khusus yang digunakan untuk menginisialisasi objek pada saat objek tersebut dibuat atau diinstansiasi. Constructor memiliki nama yang sama dengan nama kelas dan tidak memiliki tipe pengembalian. Dalam bahasa Java, ketika sebuah objek dibuat, constructor akan dipanggil secara otomatis untuk menginisialisasi nilai-nilai awal dari objek tersebut. Constructor dapat digunakan untuk mengatur nilai-nilai default dari variabel-variabel anggota (atau field) dalam kelas, dan juga untuk melakukan operasi lain yang diperlukan untuk mempersiapkan objek.

Constructor dapat memiliki parameter atau tidak memiliki parameter. Constructor yang tidak memiliki parameter disebut constructor default, dan biasanya digunakan untuk membuat objek kosong atau objek dengan nilai-nilai default. Constructor dengan parameter digunakan untuk memberikan nilai-nilai awal ke variabel-variabel anggota dari objek yang dibuat.

B. Overloading

Overloading adalah sebuah konsep dalam pemrograman di mana sebuah kelas atau sebuah method bisa memiliki beberapa implementasi yang berbeda dengan nama yang sama, namun memiliki parameter yang berbeda-beda. Artinya, dengan menggunakan overloading, kita dapat membuat beberapa method atau constructor dengan nama yang sama, tetapi memiliki parameter yang berbeda, sehingga kita dapat memanggil method atau constructor yang tepat tergantung pada jenis dan jumlah parameter yang diberikan.

Keuntungan dari overloading adalah kita dapat memudahkan penggunaan method atau constructor, karena pengguna tidak perlu mengingat nama method atau constructor yang berbeda tergantung pada jenis parameter yang digunakan. Selain itu, overloading juga dapat meminimalkan jumlah kode yang perlu ditulis, karena kita dapat menggunakan nama yang sama untuk beberapa method atau constructor yang memiliki fungsi yang sama atau mirip.

C. Constructor Overloading

Constructor overloading adalah teknik dalam bahasa pemrograman Java yang dapat membuat beberapa constructor dengan nama yang sama namun memiliki parameter yang berbeda. Dalam hal ini, setiap constructor memiliki implementasi yang berbeda tergantung pada tipe dan jumlah parameter yang diterima. Dengan constructor overloading, objek dapat dibuat dengan cara yang berbeda-beda, tergantung pada jenis parameter yang diberikan. Ini memberikan fleksibilitas dalam membuat objek dan dapat membantu menghindari duplikasi kode dalam kelas.

D. Method Overloading

Method overloading adalah konsep dalam pemrograman di mana sebuah kelas memiliki beberapa method dengan nama yang sama, tetapi memiliki parameter yang berbeda-beda. Dalam method overloading, setiap method memiliki implementasi yang berbeda tergantung pada tipe dan jumlah parameter yang diterima.

Dalam Java, method overloading memungkinkan kita untuk membuat beberapa method dengan nama yang sama, tetapi dengan jumlah atau tipe parameter yang berbeda. Hal ini memudahkan kita untuk memanggil method dengan nama yang sama, tetapi dengan parameter yang berbeda-beda tergantung pada kebutuhan aplikasi. Dalam method overloading, nama method harus sama, tetapi jumlah dan jenis parameter harus berbeda-beda. Selain itu, tipe pengembalian method dapat sama atau berbeda-beda.

E. UML

UML (Unified Modeling Language) adalah bahasa pemodelan visual yang digunakan untuk merancang dan menggambarkan struktur, perilaku, dan interaksi sistem atau aplikasi yang kompleks. Salah satu jenis diagram UML yang sering digunakan dalam pemrograman berorientasi objek adalah diagram class. Diagram class UML digunakan untuk menggambarkan struktur kelas atau objek dalam sebuah sistem atau aplikasi. Dalam diagram class UML, kelas direpresentasikan oleh sebuah kotak yang terdiri dari tiga bagian utama:

- Nama kelas: Nama kelas ditulis di bagian atas kotak, biasanya di tengah. Nama kelas harus unik dalam sebuah sistem atau aplikasi.
- Atribut kelas: Atribut kelas adalah variabel atau properti yang dimiliki oleh kelas. Atribut kelas ditulis di bagian tengah kotak, di bawah nama kelas. Setiap atribut memiliki tipe data dan visibilitas (public, private, atau protected).

- Method kelas: Method kelas adalah perilaku atau fungsi yang dimiliki oleh kelas. Method kelas ditulis di bagian bawah kotak, di bawah atribut kelas. Setiap method memiliki nama, parameter, tipe pengembalian, dan visibilitas.

Selain tiga bagian utama, ada juga beberapa elemen tambahan yang dapat ditambahkan ke diagram class UML, seperti association (hubungan antara kelas), inheritance (pewarisan), dan interface.

ALAT DAN KOMPONEN

- Komputer PC atau Laptop
- Internet

LANGKAH-LANGKAH PERCOBAAN

1. Melakukan pengaturan format jumlah angka dibelakang koma pada bilangan decimal. Modifikasi main program sebagai pengetesan pada tugas pendahuluan ke-3.

```
1  import java.text.DecimalFormat;
2  public class HitungLingkaranFormatDesimal{
3          public static void main(String args[]){
4          DecimalFormat FormatData = new DecimalFormat("#.##");
5          Lingkaran c = new Lingkaran(2.013);
6          double LuasFormatDesimal;
7          double KelilingFormatDesimal;
8          LuasFormatDesimal = Double.valueOf(FormatData.format(c.getLuas()));
9          KelilingFormatDesimal = Double.valueOf(FormatData.format(c.getKeliling()));
10         System.out.println("Luas = "+LuasFormatDesimal+" dan Keliling "+KelilingFormatDesimal);
11     }
12 }
```

Tampilan yang diinginkan untuk 2 angka dibelakang koma

```
run:
Luas = 12.72 dan Keliling 12.64
BUILD SUCCESSFUL (total time: 0 seconds)
```

Modifikasi program untuk menampilkan 3,4, dan 5 angka dibelakang koma

```
run:
Luas = 12.724 dan Keliling 12.642
BUILD SUCCESSFUL (total time: 0 seconds)
```

```
run:
Luas = 12.72381 dan Keliling 12.64164
BUILD SUCCESSFUL (total time: 0 seconds)
```

```
run:
Luas = 12.7238 dan Keliling 12.6416
BUILD SUCCESSFUL (total time: 0 seconds)
```

2. Penggunaan Konstruktor dan Metode
Mengimplementasikan UML class diagram dalam program untuk class Tabungan

Tabungan
-saldo:int
+Tabungan(initsaldo:int) +AmbilUang(jumlah:int)

Transformasikan class diagram diatas ke dalam bentuk program. Tulis listing program berikut ini sebagai pengetesan.

```java
public class CetakTabungan {
    public static void main(String args[]){
        Tabungan tabungan = new Tabungan(1200000);
        System.out.println("Saldo awal : "+tabungan.saldo);
        tabungan.AmbilUang(250000);
        System.out.println("Jumlah uang yang diambil : 250000");
        System.out.println("Saldo sekarang : "+tabungan.saldo);
    }
}
```

Tampilan yang diharapkan seperti di bawah ini.

```
run:
Saldo awal : 1200000
Jumlah uang yang diambil : 250000
Saldo sekarang : 950000
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. Overloading terhadap Konstruktor

Mengimplementasikan UML class diagram dalam program untuk class KenaikanGaji.

KenaikanGaji
-gajipokok:double -persen:double -gajibaru:double -jabatan:String -id:String
+KenaikanGaji(id:String, jabatan:String, gajipokok:double, persen:double) +KenaikanGaji(id:String, gajipokok:double , jabatan:String, persen:double) +KenaikanGaji(id:String, gajipokok:double, persen:double, jabatan:String) +getGajibaru():double +getId():String +getJabatan():String

$$gajibaru = gajipokok + (\frac{persen}{100}) * gajipokok$$

Transformasikan class diatas ke dalam bentuk program. Tulislah listing program berikut sebagai pengetesan.

```java
public class CetakKenaikanGaji {
    public static void main(String args[]){
        KenaikanGaji santi = new KenaikanGaji("SBY00034", "Manager", 4000000, 5);
        KenaikanGaji ronny = new KenaikanGaji("BDG0567", 2000000, "Staff", 7.5);
        KenaikanGaji indra = new KenaikanGaji("JKT9876", 8000000, 3.2, "Direktur");
        System.out.println("Nama : Santi");
        System.out.println("No Pegawai : "+santi.getId());
        System.out.println("Jabatan : "+santi.getJabatan());
        System.out.println("Gaji Baru : "+santi.getGajibaru());
        System.out.println("");

        System.out.println("Nama : Ronny");
        System.out.println("No Pegawai : "+ronny.getId());
        System.out.println("Jabatan : "+ronny.getJabatan());
        System.out.println("Gaji Baru : "+ronny.getGajibaru());
        System.out.println("");

        System.out.println("Nama : Indra");
        System.out.println("No Pegawai : "+indra.getId());
        System.out.println("Jabatan : "+indra.getJabatan());
        System.out.println("Gaji Baru : "+indra.getGajibaru());
        System.out.println("");
    }
}
```

Tampilan yang diharapkan seperti di bawah ini.

```
run:
Nama : Santi
No Pegawai : SBY00034
Jabatan : Manager
Gaji Baru : 4200000.0

Nama : Ronny
No Pegawai : BDG0567
Jabatan : Staff
Gaji Baru : 2150000.0

Nama : Indra
No Pegawai : JKT9876
Jabatan : Direktur
Gaji Baru : 8256000.0

BUILD SUCCESSFUL (total time: 0 seconds)
```

4. Overloading terhadap Metode

Mengimplementasikan UML class diagram dalam program untuk class Perpustakaan

Perpustakaan
-judul:String -idbuku:int -jumlahbuku:int
+Perpustakaan(jumlahbuku:int) +PeminjamanBuku(judul:String, idbuku:int) +PeminjamanBuku(idbuku:int, judul:String) +CetakPeminjaman()

Transformasikan class diagram diatas ke dalam bentuk program. Tulislah listing program berikut ini sebagai pengetesan.

```
23  public class CetakPerpustakaan {
24      public static void main(String args[]){
25          Perpustakaan anggota = new Perpustakaan(0);
26          System.out.println("===========Daftar Peminjaman Buku===========");
27          anggota.PeminjamanBuku("Dasar Pemrograman Java 2", 12098);
28          anggota.CetakPeminjaman();
29          anggota.PeminjamanBuku(23337, "Pemrograman Berorientasi Objek");
30          anggota.CetakPeminjaman();
31          System.out.println("Jumlah Buku : "+anggota.jumlahbuku);
32          System.out.println("============================================");
33      }
34  }
```

Tampilan yang diharapkan seperti di bawah ini.

```
run:
===========Daftar Peminjaman Buku===========
Judul Buku : Dasar Pemrograman Java 2
ID Buku : 12098
Judul Buku : Pemrograman Berorientasi Objek
ID Buku : 23337
Jumlah Buku : 2
============================================
BUILD SUCCESSFUL (total time: 0 seconds)
```

5. Tugas Resmi

1) Analisa dan berikan kesimpulan pada percobaan yang sudah dilakukan.
2) Jelaskan apa yang dimaksud dengan overloading pada konstruktor dan metode?
3) Buat UML class diagram yang didalamnya digunakan konstruktor dan metode serta implementasi dalam bentuk program. Selanjutnya buat main program untuk melakukan pengetesan dari program yang sudah dibuat.
4) Buat UML class diagram yang didalamnya digunakan overloading terhadap constructor atau overloading terhadap metode (pilih salah satu), serta implementasikan dalam bentuk program. Selanjutnya buat main program untuk melakukan pengetesan dari program yang sudah dibuat.

HASIL PRAKTIKUM

1. Program 1

```
import java.text.DecimalFormat;

public class HitungLingkaranFormatDesimal{
        public static void main(String args[]){
                DecimalFormat FormatData = new DecimalFormat("#.##");
                Lingkaran c = new Lingkaran(2.013);
                double LuasFormatDesimal;
                double KelilingFormatDesimal;
                LuasFormatDesimal = Double.valueOf(FormatData.format(c.getLuas()));
                KelilingFormatDesimal = Double.valueOf(FormatData.format(c.getKeliling()));
                System.out.println("Luas = "+LuasFormatDesimal+ " dan Keliling " +KelilingFormatDesimal);
        }
}
class Lingkaran{
        private double r;
        public Lingkaran(double er){
                this.r = er;
        }
        public double getLuas(){
                return(3.14*r*r);
        }
        public double getKeliling(){
                return(2*3.14*r);
        }
}
```

- **2 digit dibelakang koma**

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac HitungLingkaranFormatDesimal.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java HitungLingkaranFormatDesimal
Luas = 12.72 dan Keliling 12.64
```

- **3 digit dibelakang koma**

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac HitungLingkaranFormatDesimal.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java HitungLingkaranFormatDesimal
Luas = 12.724dan Keliling 12.642
```

- **4 digit dibelakang koma**

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac HitungLingkaranFormatDesimal.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java HitungLingkaranFormatDesimal
Luas = 12.7238 dan Keliling 12.6416
```

- **5 digit dibelakang koma**

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac HitungLingkaranFormatDesimal.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java HitungLingkaranFormatDesimal
Luas = 12.72381 dan Keliling 12.64164
```

2. Program 2

```
class Tabungan{
        private int saldo;
        public Tabungan(int initsaldo){
                saldo = initsaldo;
        }
        public void AmbilUang(int jumlah){
                saldo = saldo-jumlah;
        }
        public int getSaldo(){
                return(saldo);
        }
}
public class CetakTabungan{
        public static void main(String args[]){
                Tabungan tabungan = new Tabungan(1200000);
                System.out.println("Saldo awal: " +tabungan.getSaldo());
                tabungan.AmbilUang(250000);
                System.out.println("Jumlah uang yang diambil: 250000");
                System.out.println("Saldo sekarang: "+tabungan.getSaldo());
        }
}
```

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac CetakTabungan.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java CetakTabungan
Saldo awal: 1200000
Jumlah uang yang diambil: 250000
Saldo sekarang: 950000
```

3. Program 3

```
class KenaikanGaji{
        private double gajipokok;
        private double persen;
        private double gajibaru;
        private String jabatan;
        private String id;

        public KenaikanGaji(String id1, String jabatan1, double gajipokok1, double persen1){
                id = id1;
                jabatan = jabatan1;
                gajipokok = gajipokok1;
                persen = persen1;
        }
        public KenaikanGaji(String id2, double gajipokok2, String jabatan2, double persen2){
                id = id2;
                jabatan = jabatan2;
                gajipokok = gajipokok2;
                persen = persen2;
        }
        public KenaikanGaji(String id3, double gajipokok3, double persen3, String jabatan3){
                id = id3;
                jabatan = jabatan3;
                gajipokok = gajipokok3;
                persen = persen3;
        }
        public double getGajibaru(){
                gajibaru = gajipokok + ((persen/100)*this.gajipokok);
                return(gajibaru);
        }
        public String getId(){
                return(id);
        }
        public String getJabatan(){
                return(jabatan);
        }
}
```

```java
public class CetakKenaikanGaji{
        public static void main (String args[]){
                KenaikanGaji santi = new KenaikanGaji("SBY00034","Manager",4000000,5);
                KenaikanGaji ronny = new KenaikanGaji("BDG0567",2000000,"Staff",7.5);
                KenaikanGaji indra = new KenaikanGaji("JKT9876",8000000,3.2,"Direktur");
                System.out.println("Nama : Santi");
                System.out.println("No Pegawai : "+santi.getId());
                System.out.println("Jabatan: "+santi.getJabatan());
                System.out.println("Gaji Baru: "+santi.getGajibaru());
                System.out.println("");

                System.out.println("Nama : Ronny");
                System.out.println("No Pegawai : "+ronny.getId());
                System.out.println("Jabatan: "+ronny.getJabatan());
                System.out.println("Gaji Baru: "+ronny.getGajibaru());
                System.out.println("");

                System.out.println("Nama : Indra");
                System.out.println("No Pegawai : "+indra.getId());
                System.out.println("Jabatan: "+indra.getJabatan());
                System.out.println("Gaji Baru: "+indra.getGajibaru());
                System.out.println("");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac CetakKenaikanGaji.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java CetakKenaikanGaji
Nama : Santi
No Pegawai : SBY00034
Jabatan: Manager
Gaji Baru: 4200000.0

Nama : Ronny
No Pegawai : BDG0567
Jabatan: Staff
Gaji Baru: 2150000.0

Nama : Indra
No Pegawai : JKT9876
Jabatan: Direktur
Gaji Baru: 8256000.0
```

4. Program 4

```
class Perpustakaan{
        private String judul;
        private int idbuku;
        private int jumlahbuku;

        public Perpustakaan(int jumlahbuku0){
                jumlahbuku = jumlahbuku0;
        }
        public void PeminjamanBuku(String judul1, int idbuku1){
                judul = judul1;
                idbuku = idbuku1;
        }
        public void PeminjamanBuku(int idbuku2, String judul2){
                idbuku = idbuku2;
                judul = judul2;
        }
        public void CetakPeminjaman(){
                System.out.println("Judul Buku: "+judul);
                System.out.println("ID Buku: "+idbuku);
        }
        public int getJumlahbuku(){
                return(jumlahbuku);
        }
}
public class CetakPerpustakaan{
        public static void main(String args[]){
                Perpustakaan anggota = new Perpustakaan(0);
                System.out.println("==================== Daftar Peminjaman Buku
====================");
                anggota.PeminjamanBuku("Dasar Pemrograman Java 2", 12098);
                anggota.CetakPeminjaman();
                anggota.PeminjamanBuku(23337,"Pemrograman Berorientasi Objek");
                anggota.CetakPeminjaman();
                System.out.println("Jumlah Buku: "+anggota.getJumlahbuku());
                System.out.println
("================================================================");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac CetakPerpustakaan.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java CetakPerpustakaan
==================== Daftar Peminjaman Buku ====================
Judul Buku: Dasar Pemrograman Java 2
ID Buku: 12098
Judul Buku: Pemrograman Berorientasi Objek
ID Buku: 23337
Jumlah Buku: 0
================================================================
```

HASIL TUGAS RESMI

1. **Program 5**

Info
+main()
DiriSendiri
-nama :String
+ DiriSendiri(nama1 :String) + getNama() :String

```
class DiriSendiri{
        private String nama;

        public DiriSendiri(String nama1){
                nama = nama1;
        }
        public String getNama(){
                return(nama);
        }
}
public class Info{
        public static void main(String args[]){
                DiriSendiri pemilik = new DiriSendiri("Elsadanika");
                System.out.println("=================== Informasi ==================");
                System.out.println("Nama: "+pemilik.getNama());
                System.out.println
("============================================================");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac Info.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java Info
=================== Informasi ==================
Nama: Elsadanika
============================================================
```

2. **Program 6**

CetakRekening
+main()

Rekening
-nama :String -password :int -cardnumber :int
+ Rekening(cardnumber1 :int) + PemilikRekening(nama1 :String, password1 :int) + CetakPemilik() + getCardNumber() :int

```
class Rekening{
        private String nama;
        private int password;
        private int cardnumber;

        public Rekening(int cardnumber1){
                cardnumber = cardnumber1;
        }
        public void PemilikRekening(String nama1, int password1){
                nama = nama1;
                password = password1;
        }
        public void CetakPemilik(){
                System.out.println("Nama: "+nama);
                System.out.println("Password: "+password);
        }
        public int getCardNumber(){
                return(cardnumber);
        }
}
public class CetakRekening{
        public static void main(String args[]){
                Rekening pemilik = new Rekening(312751124);
                System.out.println("================== Informasi Pribadi
==================");
                pemilik.PemilikRekening("Elsadanika Maura",112233);
                pemilik.CetakPemilik();
                System.out.println("Card Number: "+pemilik.getCardNumber());
                System.out.println
("============================================================");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\ConstNOv>javac CetakRekening.java

D:\KULIAH SMT 4\APKOM\ConstNOv>java CetakRekening
================== Informasi Pribadi ==================
Nama: Elsadanika Maura
Password: 112233
Card Number: 312751124
============================================================
```

PEMBAHASAN

Pada program pertama, diketahui bahwa terdapat class HitungLingkaranFormatDesimal yang memiliki main method dengan rincian statements seperti tertera pada bagian hasil praktikum. Program ini memiliki konstruktor dalam class dan memberi nilai return dari method dalam class Lingkaran. Digunakan paket Java dalam proses impor kelas DesimalFormat. Selain itu, data pada program ini dikonversi dari String ke double melalui perintah Double.valueOf(). Jari-jari pada lingkaran dideklarasikan pada class HitungLingkaranFormatDesimal dengan variable c dan parameter 2.013 sebagai jari-jarinya. Nilai jari-jari ini dipanggil pada class Lingkaran, karena aksesnya adalah private maka dipanggil melalui konstruktor. Digunakan pula method getLuas dan getKeliling untuk memberikan nilai return hasil perhitungan dari class Lingkaran ke class HitungLingkaranFormatDesimal. Sementara itu, untuk mendapatkan nilai dibelakang koma sejumlah 2, 3, 4, atau 5 maka yang perlu dilakukan adalah menambah symbol pagar (#) sebanyak jumlah digit dibelakang koma yang diinginkan.

Pada program kedua, yaitu CetakTabungan, diketahui bahwa class ini memiliki informasi mengenai saldo awal, jumlah uang yang diambil, dan saldo saat ini untuk dicetak ke layar. Proses mencetak saldo ke monitor dilakukan melalui objek yang dibentuk pada class Tabungan. Dalam class Tabungan terdapat variabel saldo, konstruktor Tabungan, method AmbilUang, dan method getSaldo. Konstruktor Tabungan digunakan untuk menginisialisasi variabel saldo dengan nilai awal yang diberikan. Method AmbilUang digunakan untuk proses kalkulasi mengurangi saldo sebesar jumlah yang diberikan. Sedangkan method getSaldo digunakan untuk memberi nilai return saldo saat ini. Program ini memiliki konsep objek oriented yang ditunjukkan pada kelas Tabungan yang dibuat sebagai cetak biru dari objek tabungan yang akan dibuat.

Pada program ketiga, terdapat dua class yaitu class KenaikanGaji dan CetakKenaikanGaji. Class KenaikanGaji memiliki empat atribut yaitu gajipokok, persen, gajibaru, jabatan, dan id. Dibentuk konstruktor KenaikanGaji sebanyak 3 buah untuk membedakan atribut-atribut untuk masing-masing objek. Selain itu pada class ini terdapat method getGajibaru untuk mengkalkulasi gaji baru yang didapatkan setiap objek pada class CetakKenaikanGaji dan memberikan nilai return berupa gajibaru. Method getId dan getJabatan pun memberikan returnnya berupa id dan jabatan.

Diketahui bahwa terdapat 3 objek yang dibentuk pada class CetakKenaikanGaji, yaitu santi, ronny, dan indra. Diberikan parameter bertipe String, int, dan double yang disusun berbeda-beda. Parameter ini dipanggil melalui konstruktor KenaikanGaji pada class KenaikanGaji. Lalu dicetak pada class CetakKenaikanGaji melalui pemanggilan method getId, getJabatan, dan getGajibaru yang disesuaikan dengan objek-objek tersebut.

Hampir serupa dengan program ketiga, program keempat memiliki dua class yaitu Perpustakaan dan CetakPerpustakaan. Terdapat tiga atribut bersifat private pada class Perpustakaan yaitu judul, idbuku, dan jumlahbuku. Method PeminjamanBuku memiliki dua jenis input yang diterima yaitu judul dan idbuku. Method CetakPeminjaman digunakan untuk mencetak judul dan idbuku dari buku yang dipinjam. Sedangkan method getJumlahbuku digunakan untuk mengambil data jumlahbuku yang tersedia. Pada main program, dibuat objek anggota dari kelas Perpustakaan dengan inisiasi jumlahbuku awal sebanyak 0. Kemudian dilakukan pemanggilan method PeminjamanBuku dan input yang diberikan adalah judul buku dan idbuku. Setelah itu dilakukan pemanggilan method CetakPeminjaman untuk mencetak judul dan idbuku dari buku yang dipinjam. Hal yang sama dilakukan untuk peminjaman buku kedua dengan memanggil method PeminjamanBuku dan input yang diberikan adalah idbuku dan judul buku. Kemudian dilakukan pemanggilan method CetakPeminjaman lagi untuk mencetak judul dan idbuku dari buku yang dipinjam. Terakhir, dilakukan pemanggilan method getJumlahbuku untuk mencetak jumlah buku yang tersedia di perpustakaan.

Overloading pada konstruktor dan method merupakan kondisi untuk membuat beberapa definisi konstruktor dengan nama yang sama namun parameternya berbeda. Hal ini memungkinkan untuk mempergunakan nama yang sama untuk tugas yang berbeda atau untuk memanggil konstruktor dengan parameter yang berbeda. Overloading konstruktor memungkinkan untuk membuat beberapa konstruktor dalam satu class dengan parameter berbeda yang memudahkan proses pembuatan objek tanpa perlu membuat kelas yang berbeda. Ketika objek dibuat, konstruktor yang sesuai akan dipanggil tergantung pada parameter yang digunakan.

Overloading method merupakan kondisi untuk membuat beberapa definisi method dengan nama yang sama namun parameter berbeda. Method yang dibentuk memiliki nama yang sama dengan parameter berbeda namun masih dalam satu class yang sama.

Hal ini memudahkan untuk melakukan tugas yang sama dengan cara yang berbeda tanpa perlu membuat banyak method dengan nama yang berbeda. Ketika method dipanggil, maka method yang sesuai akan dipanggil tergantung pada parameter yang digunakan.

KESIMPULAN

Melalui praktikum yang telah dilakukan, diketahui bahwa constructor meripakan metode untuk menginisialisasi objek pada saat objek dibuat. Contruktor memiliki nama yang sama dengan nama class akan tetapi tidak memiliki nilai return. Construktor dapat memiliki atau tidak memiliki parameter. Sedangkan overloading merupakan konsep class atau method yang bisa memilki beberapa implementasi berbeda dengan nama yang sama namun parameter yang berbeda-beda. Overloading constructor memungkinkan untuk membuat beberapa konstruktor dalam satu class dengan parameter berbeda yang memudahkan proses pembuatan objek tanpa perlu membuat kelas yang berbeda. Ketika objek dibuat, konstruktor yang sesuai akan dipanggil tergantung pada parameter yang digunakan. Overloading method merupakan kondisi untuk membuat beberapa definisi method dengan nama yang sama namun parameter berbeda. Method yang dibentuk memiliki nama yang sama dengan parameter berbeda namun masih dalam satu class yang sama.

Diagram class UML, kelas direpresentasikan oleh sebuah kotak yang terdiri dari tiga bagian utama:

- Nama kelas: Nama kelas ditulis di bagian atas kotak, biasanya di tengah. Nama kelas harus unik dalam sebuah sistem atau aplikasi.
- Atribut kelas: Atribut kelas adalah variabel atau properti yang dimiliki oleh kelas. Atribut kelas ditulis di bagian tengah kotak, di bawah nama kelas. Setiap atribut memiliki tipe data dan visibilitas (public, private, atau protected).
- Method kelas: Method kelas adalah perilaku atau fungsi yang dimiliki oleh kelas. Method kelas ditulis di bagian bawah kotak, di bawah atribut kelas. Setiap method memiliki nama, parameter, tipe pengembalian, dan visibilitas.

Selain tiga bagian utama, ada juga beberapa elemen tambahan yang dapat ditambahkan ke diagram class UML, seperti association (hubungan antara kelas), inheritance (pewarisan), dan interface.

DAFTAR PUSTAKA

[1] Softscients. (2021). Jenis Simbol Operator. [softscients.com]. Available: https://softscients.com/2020/03/25/jenis-operator/

[2] Muhardian, A. (2016). Apa Fungsi String[] args pada Pemrograman Java?. [petanikode.com]. Available: https://www.petanikode.com/java-args/

[3] Redaksi Jagoan Hosting. (2022). Ap aitu Java? Ini Pengerttian, Fungsi, Cara Kerja & Contoh. [jagoanhosting.com]. Available: https://www.jagoanhosting.com/blog/java-adalah/